

Computer Science & IT

Data Structure & programming



Queue

Lecture No. 02



By- Abhishek Sir

Recap of Previous Lecture



Topic

Linear Queue

Topic

Enqueue

Topic

Dequeue

Topic

Circular Queue

Topic

Overflow Condition

Topics to be Covered



Topic

Circular queue

Topic

Queue using stack

Topic

Deque

Topic

priority queue.

Topic

overflow

if $((front == 0 \ \&\& \ rear == Max - 1) \ || \ (front == rear + 1))$

$f = 0, r = 4$

f.	r
1	0
2	1
3	2
4	3

if $(front == (rear + 1) \% Max)$

Max
= 5



Topic : Circular Queue

```
void Enqueue(int data){  
    if (front == (rear+1)%Max){  
        printf("Queue is full");  
        return;  
    }  
    rear = (rear+1)%Max;  
    arr[rear] = data;  
    if (front == -1)  
        front = 0;  
}
```



Topic : Circular Queue

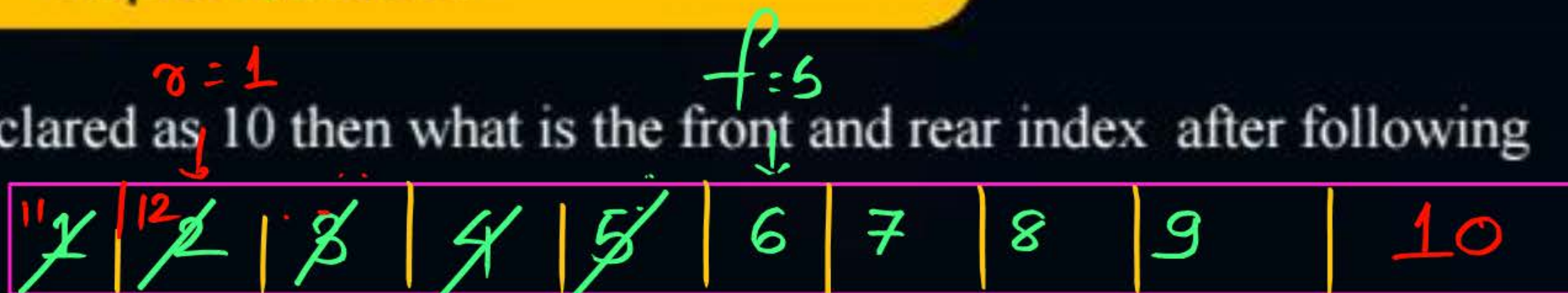


```
int dequeue() {  
    int data;  
    if (isEmpty()) {  
        printf("Queue is Empty");  
        return -1;  
    }  
    data = arr[front];  
    if (front == rear)  
        front = rear = -1;  
    else  
        front = (front + 1) % Max;  
    return data;  
}
```




Topic: Question

Q. If the Max is declared as 10 then what is the front and rear index after following operation



I: Enqueue(1), Enqueue(2), Enqueue(3) Enqueue(4) Enqueue(5) Dequeue(),
Dequeue(), Dequeue()

$$w = 3, x = 4$$

II: Enqueue(6), Enqueue(7), Enqueue(8) Enqueue(9) Dequeue(), Dequeue()
Enqueue(10), Enqueue(11), Enqueue(12) $y = 5, z = 1$

if after Operation I front and rear value is w, x and After completion of II the front and rear value is y and z then value of $w + 10x - 2y + 4z$ is [37]

$$3 + 10 \times 4 - 2 \times 5 + 4 \times 1$$
$$3 + 40 - 10 + 4 = 37$$



Topic: Question

If circular queue is implemented using array size 100 with base address 500 and size as 2B following operation are performed

Circular queue

I: Enqueue 1 to 95 in reverse order.

II: Dequeue 5 elements, ✓

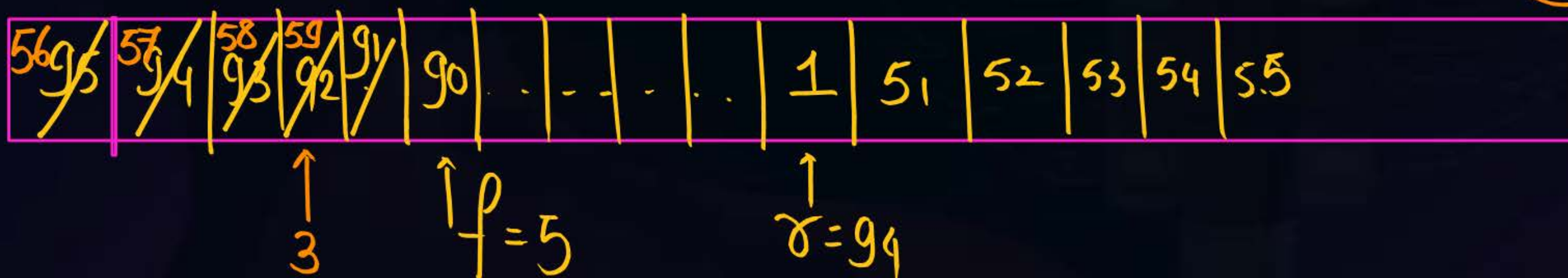
III. Enqueue 51 to 59 in given order

$$X = 500 + 5 \times 2 = 510$$

$$Y = 500 + 3 \times 2 = 506$$

If Address of front is X and address of rear is Y then $Y - X$ is

$$506 - 510 = \textcircled{-4}$$





Implementation of Queue Using Stack

$$\text{Enqueue}(x) = \text{push}(S_1, x)$$



Stack S1



Stack S2



Implementation of Queue Using Stack

Enqueue(1)



Stack S1



Stack S2



Implementation of Queue Using Stack

Enqueue(2)



Stack S1



Stack S2



Implementation of Queue Using Stack

Enqueue(3)



Stack S1



Stack S2



Implementation of Queue Using Stack



Enqueue(4)



Stack S1



Stack S2



Deque

Deque $\neq$ pop

1 pop from S_1 and push the same element S_2



Stack S1



Stack S2



Implementation of Queue Using Stack



Dequeue



Stack S1



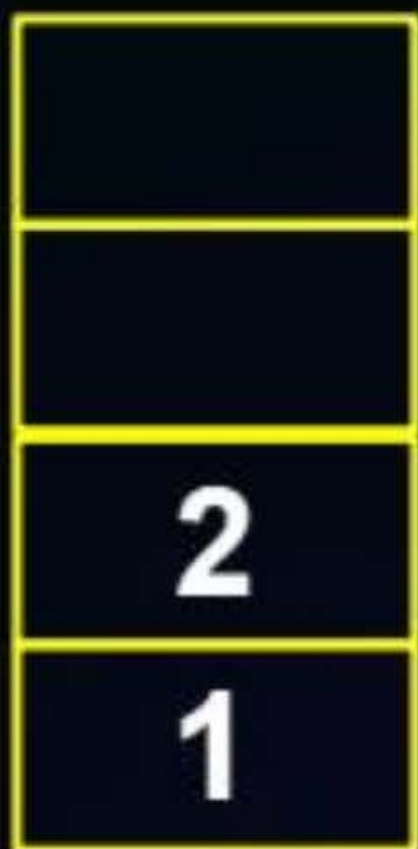
Stack S2



Implementation of Queue Using Stack



Dequeue



Stack S1



Stack S2



Implementation of Queue Using Stack



Dequeue



Stack S1



Stack S2



Deque

2. pop 1 element from S_2



Stack S1



Stack S2



Dequeue



Stack S1



Stack S2

3. Restore
pop element from S_2 and
push the same in S_1



Implementation of Queue Using Stack



Restore



Stack S1



Stack S2



Implementation of Queue Using Stack



Restore



Stack S1



Stack S2



Implementation of Queue Using Stack



Restore



Stack S1



Stack S2



Implementation of Queue Using Stack



Restore



Stack S1



Stack S2



Implementation of Queue Using Stack



Stack S1



Stack S2

1. pop from S_1 and push the same in S_2
2. pop 1 element from S_2
3. Restore pop all elements from S_2 and push in S_1 .

Question



In the above implementation of queue if 10 Enqueue and 2 dequeue operation performed then total number of push and pop done is _____

10 Enqueue : 10 push

1 $\forall x$ pop(x) from s_1 - 10 pop
push(x) in s_2 - 10 push

1 pop

$\forall x$ pop(x) s_2 - 9 pop
push(x) s_1 - 9 pop

2nd queue

1. 9 pop, 9 push

2. 1 pop

3. 8 pop, 8 push

49
35

84 Ans

Question



In the above implementation of queue if 10 Enqueue and 2 dequeue operation performed then total number of push and pop done is _____

n - Enqueue - n push

1 - n pop(s_1), n push(s_2)

2. 1 pop(s_2)

3 Restore $n-1$ pop(s_2)
 $n-1$ push(s_1)

Enqueue - n push

1 Dequeue $2n$ pop

$2n-1$ push

2nd dequeue $n=9$

18 pop

17 push
35

10 push

1 dequeue

20 pop
19 push
39

$$10 + 39 + 35 = 84$$



Question



In the above implementation of queue if 10 Enqueue and 2 multidequeue operation performed then total number of push and pop done is _____

multidequeue

n Enqueue : n push

$2n$ pop

$2n - k$ push

1. n pop(s_1)
 n push(s_2)

2. Multidequeue(k)
 k pop

3. Restore $n - k$ pop(s_2)
 $n - k$ push(s_1)

$$10 + 20 + 18 = \textcircled{48}$$



Question



In the above implementation of queue if 50 Enqueue operation and 2 dequeue operation performed. If total number of push is x and total number pop is y then $x+y$ is

dequeue.

Enqueue : 50 push

1 dequeue : $2n$ pop = 100

$2n-1$ push = 99

$n=49$ 2 dequeue : $2 \times 49 = 98$

$2 \times 49 - 1 = 97$

4.44 Answer



Question



In the above implementation of queue if 50 Enqueue operation and 2 multi dequeue operation performed. If total number of push is (x) and total number pop is (y) then $x+y$ is

multideque

$$k=2$$

$$\text{Enqueue} = n \text{ push} : 50 \text{ push}$$

$$\text{No. of pop} = 2n = 100$$

$$\text{No. of push} = 2n - k = 100 - 2 = 98$$

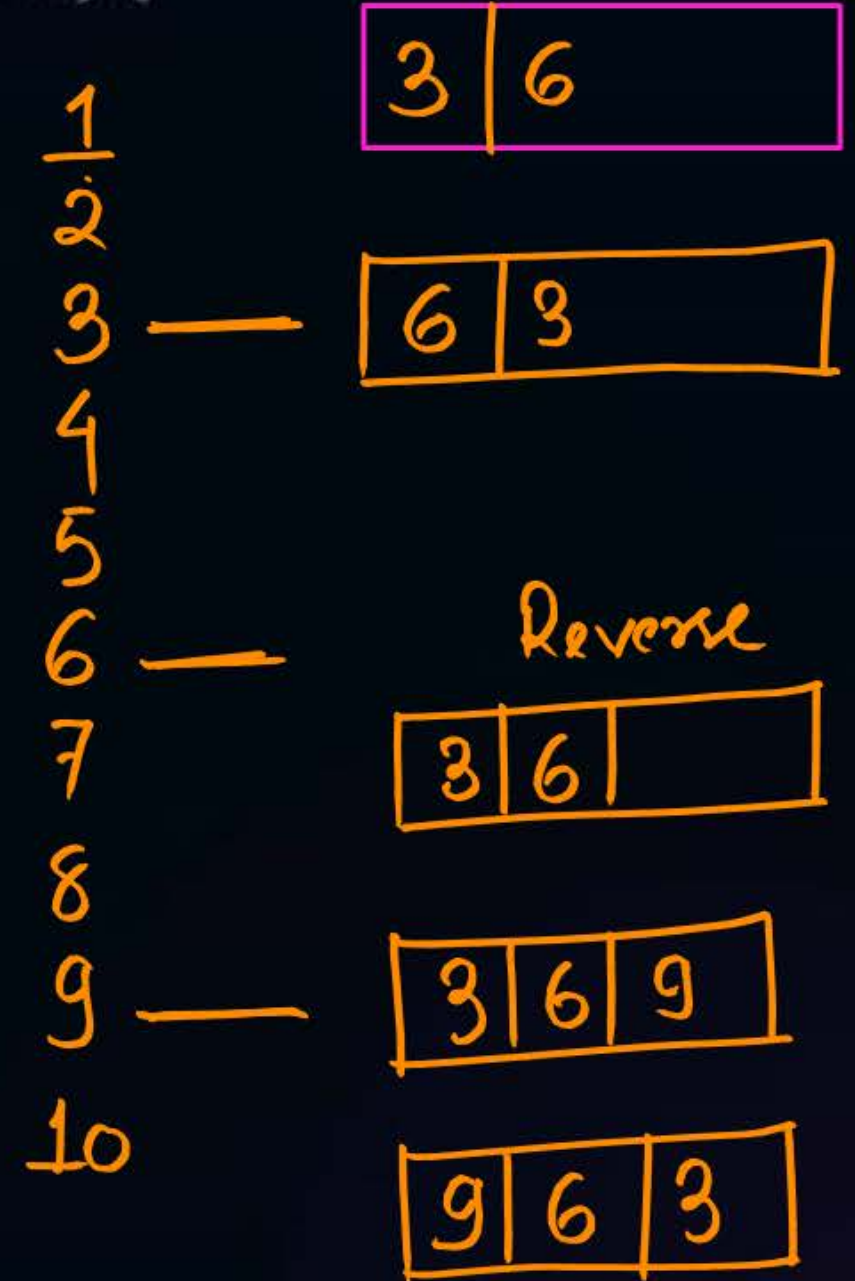
$$(248)$$

Suppose the stack S support the operation of push pop and Reverse. Reverse operation reverse the entire content of stack. Similarly Queue support the operation of Enqueue , dequeue and reverse(reverse entire content of the queue. In addition to that it also support the operation of isEmpty() for stack and queue which returns true if stack or queue is empty.

Consider the following pseudo code

The output of the program is 1*9 = 9

```
int doAnotherThing() {
    int i, x, y;
    for (i = 1; i <= 10; i++) {
        if (i % 3 == 0) {
            reverse(q); ✓
            enqueue(q, i);
            reverse(q);
        } else {
            reverse(s);
            push(s, i);
            reverse(s);
        }
    }
    x = pop(s);
    y = dequeue(q);
    return x * y;
}
```



Suppose the stack S support the operation of push pop and Reverse. Reverse operation reverse the entire content of stack. Similarly Queue support the operation of Enqueue , dequeue and reverse(reverse entire content of the queue. In addition to that it also support the operation of isEmpty() for stack and queue which returns true if stack or queue is empty.

Consider the following pseudo code

The output of the program is _____



```

int doAnotherThing() {
    int i, x, y;
    for (i = 1; i <= 10; i++) {
        if (i % 3 == 0) {
            reverse(q);
            enqueue(q, i);
            reverse(q);
        } else {
            reverse(s);
            push(s, i);
            reverse(s);
        }
    }
    x = pop(s);
    y = dequeue(q);
    return x * y;
}
    
```

1
2
3
4
5
6
7
8
9
10



Q. Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions. Which one of the following statements is TRUE with respect to this modified stack?

(GATE_2014_2 M)

out of Box

Enqueue

— Reverse

push

Reverse

Dequeue

pop

(A) A queue cannot be implemented using this stack. ~~X~~

(B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions. ~~X~~

(C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction. ✓

(D) A queue can be implemented where both ENQUEUE and DEQUEUE take ~~a~~ single instruction

Enque(1)

1

Enque(2)

2
1

Enque(3)

1
2

2
1

Reverse

3
2
1

push

1
2
3

Reverse

Deque = pop



Q. Let Q denote a queue containing 3 numbers and S be an empty stack. Head (Q) returns the element at the head of the queue Q without removing it from Q. Similarly Top (S) returns the element at the top of S without removing it from S. Consider the algorithm given below.

while Q is not Empty do

 if S is Empty OR $\text{Top}(S) \leq \text{Head}(Q)$ then

$X := \text{Dequeue}(Q)$;

 Push (S, x) ;

 else

$X := \text{Pop}(S)$;

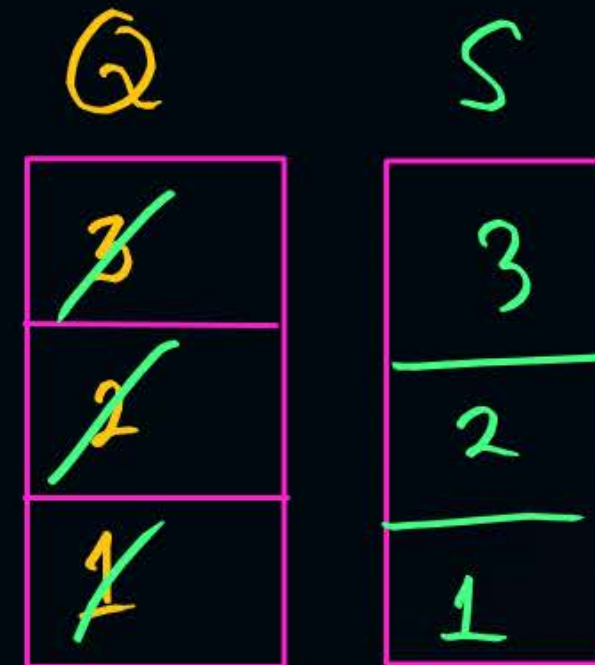
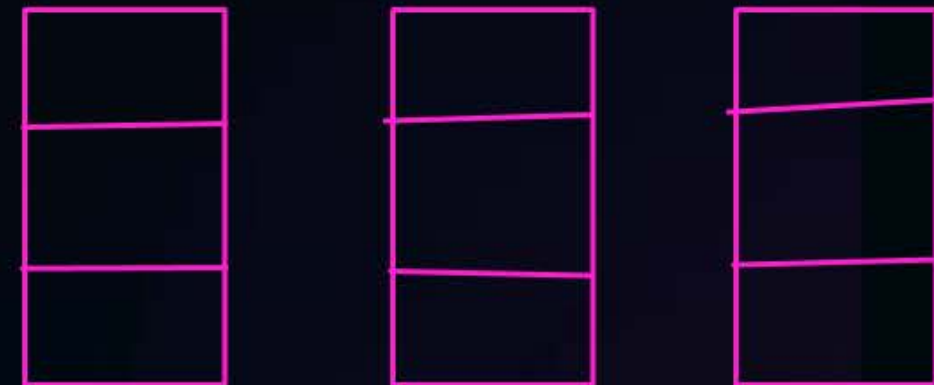
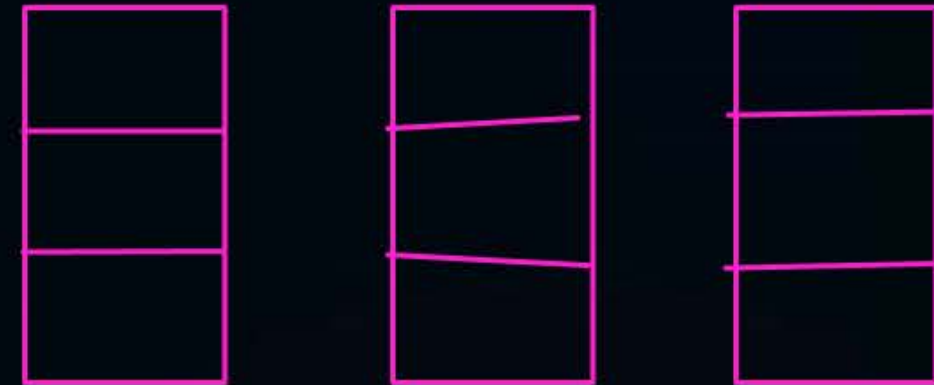
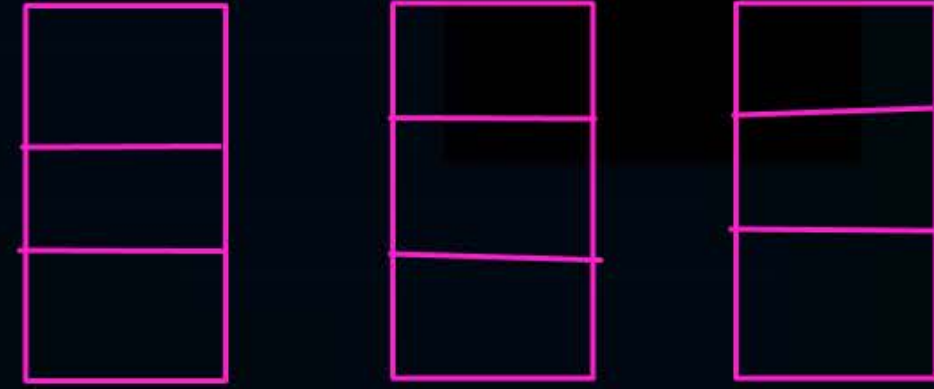
 Enqueue (Q, X) ;

 end

end

The maximum possible number of iterations of the while loop in the algorithm is _____

(GATE_2016_2 M)



Q. Let Q denote a queue containing 3 numbers and S be an empty stack. Head (Q) returns the element at the head of the queue Q without removing it from Q. Similarly Top (S) returns the element at the top of S without removing it from S. Consider the algorithm given below.

while Q is not Empty do

if S is Empty OR $\text{Top}(S) \leq \text{Head}(Q)$ then

$X := \text{Dequeue}(Q);$

Push (S, x);

else

$X := \text{Pop}(S);$

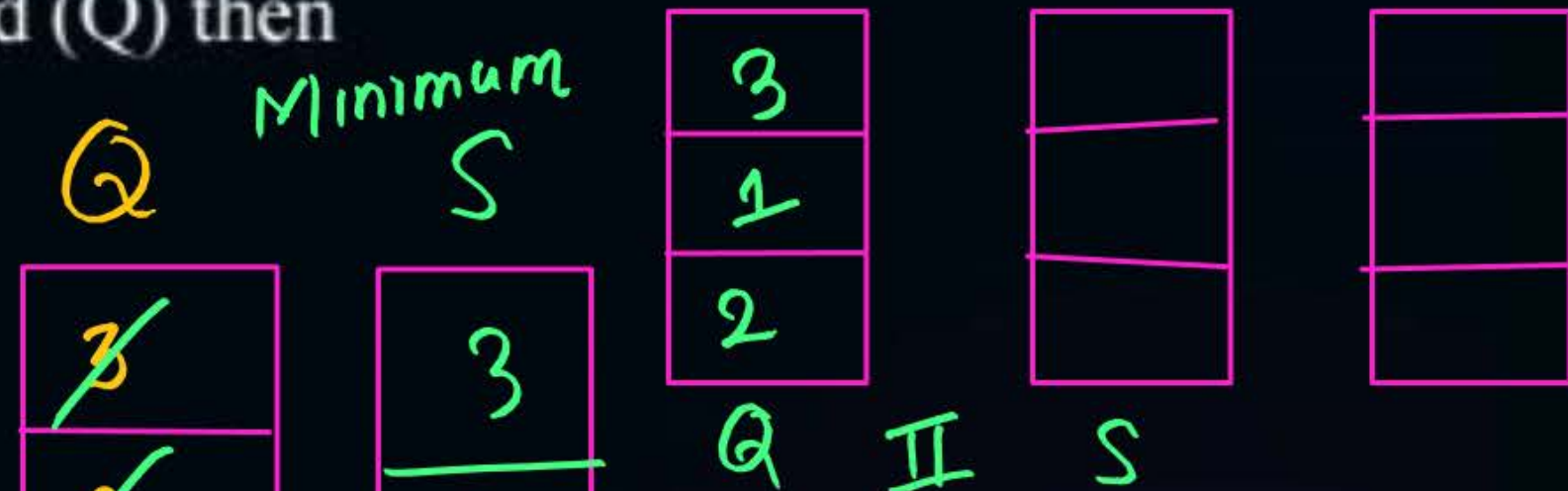
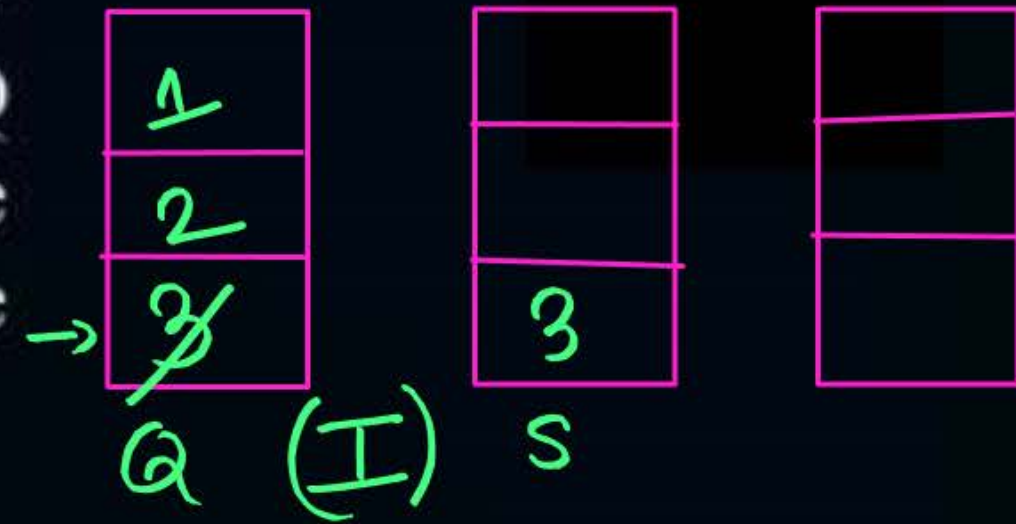
Enqueue (Q, X);

end

end

The maximum possible number of iterations of the while loop in the algorithm is _____

(GATE_2016_2 M)



Q. Let Q denote a queue containing 3 numbers and S be an empty stack. Head (Q) returns the element at the head of the queue Q without removing it from Q. Similarly Top (S) returns the element at the top of S without removing it from S. Consider the algorithm given below.

while Q is not Empty do

 if S is Empty OR Top (S) $\leq$ Head (Q) then

 X := Dequeue (Q) ;

 Push (S, x) ;

 else

 X := Pop (S) ;

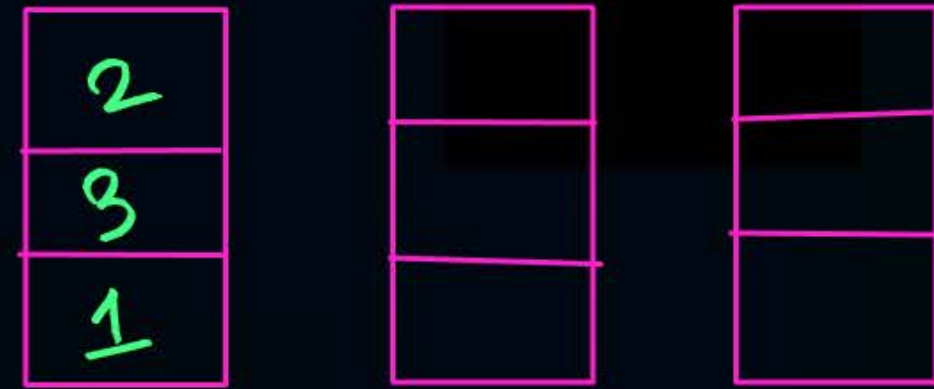
 Enqueue (Q, X) ;

end

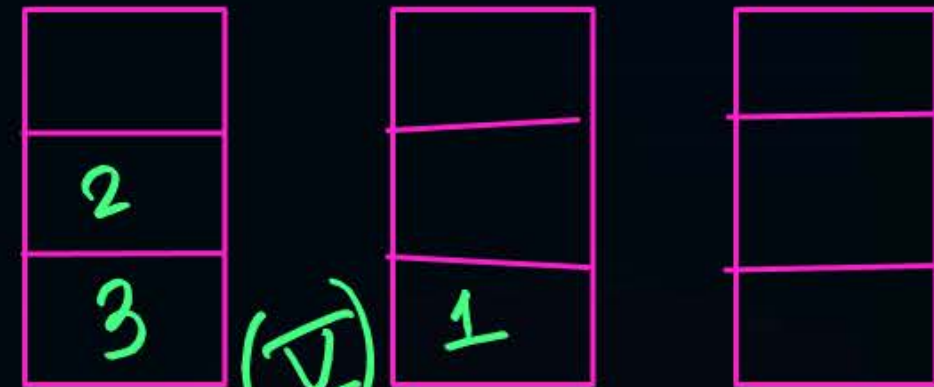
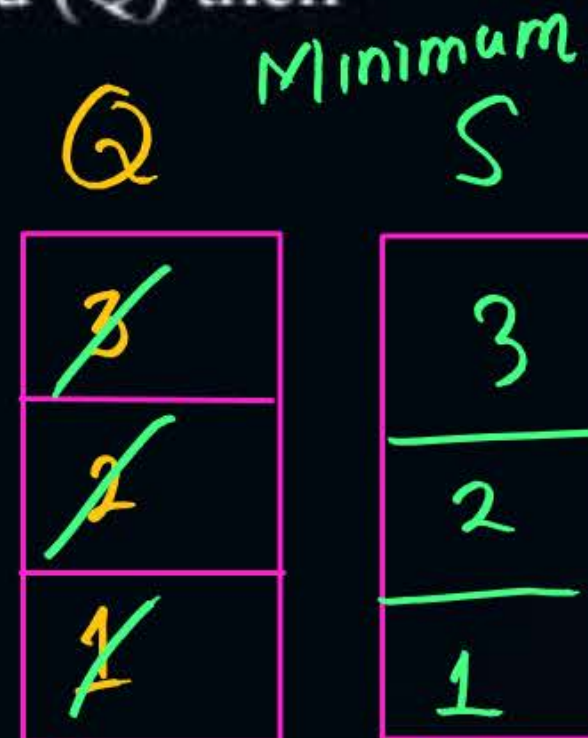
end

The maximum possible number of iterations of the while loop in the algorithm is _____

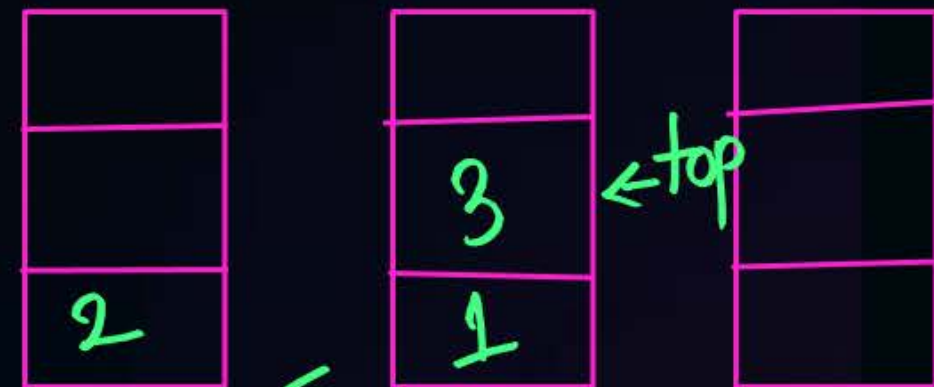
(GATE_2016_2 M)



IV



(V)



VI

Q. Let Q denote a queue containing 3 numbers and S be an empty stack. Head (Q) returns the element at the head of the queue Q without removing it from Q. Similarly Top (S) returns the element at the top of S without removing it from S. Consider the algorithm given below.

while Q is not Empty do

 if S is Empty OR Top (S) $\leq$ Head (Q) then

 X := Dequeue (Q) ;

 Push (S, x) ;

 else

 X := Pop (S) ;

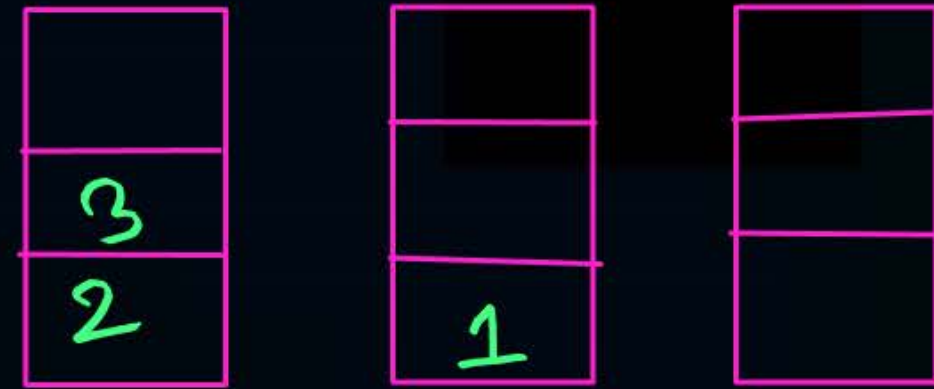
 Enqueue (Q, X) ;

 end

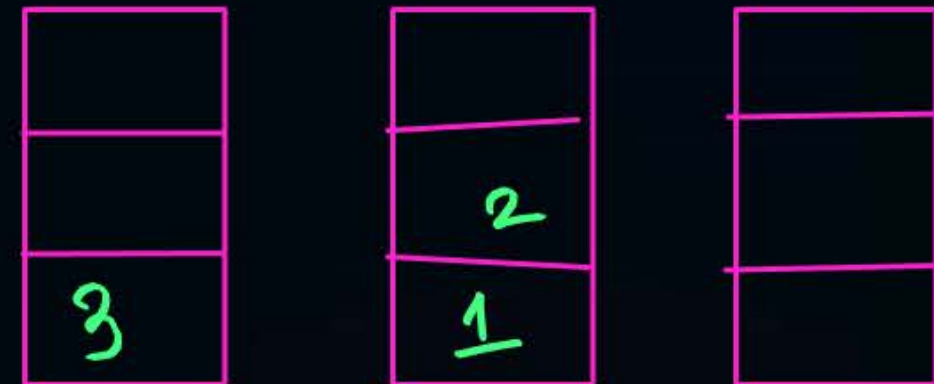
end

The maximum possible number of iterations of the while loop in the algorithm is 9

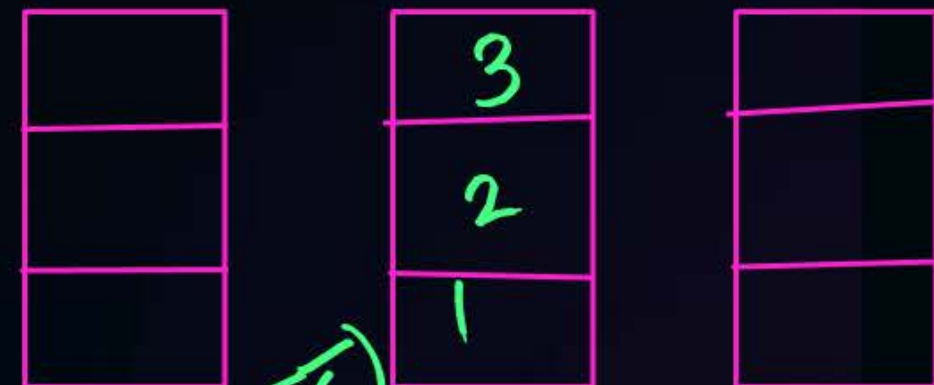
(GATE_2016_2 M)



VII



VIII



IX

THANK - YOU